Latest updates: https://dl.acm.org/doi/10.1145/3658271.3658342

RESEARCH-ARTICLE

Generating and Reviewing Programming Codes with Large Language Models: A Systematic Mapping Study

BEATRIZ VENTORINI LINS DE ALBUQUERQUE, Federal University of the State of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

ANTONIO FERNANDO SOUZA DA CUNHA, Federal University of the State of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

LEONARDO SOUZA, Federal University of the State of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

SEAN WOLFGAND MATSUI SIQUEIRA, Federal University of the State of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

RODRIGO PEREIRA DOS SANTOS, Federal University of the State of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

Open Access Support provided by:

Federal University of the State of Rio de Janeiro

PDF Download
3658271.3658342.pdf
07 April 2026
Total Citations: 3
Total Downloads: 802

Published: 23 May 2024

Citation in BibTeX format

SBSI '24: XX Brazilian Symposium on Information Systems
May 20 - 23, 2024
Juiz de Fora, Brazil

SBSI '24: Proceedings of the 20th Brazilian Symposium on Information Systems (May 2024)
https://doi.org/10.1145/3658271.3658342
ISBN: 9798400709968

Generating and Reviewing Programming Codes with Large Language Models

A Systematic Mapping Study

Beatriz Ventorini Lins de Albuquerque
Universidade Federal do Estado do Rio de Janeiro (UNIRIO)
Petrobras - Petróleo Brasileiro S.A.
Rio de Janeiro, Rio de Janeiro, Brazil
beatriz.ventorini@edu.unirio.br

Antonio Fernando Souza da Cunha
Universidade Federal do Estado do Rio de Janeiro (UNIRIO)
Petrobras - Petróleo Brasileiro S.A.
Rio de Janeiro, Rio de Janeiro, Brazil
antoniofernando@edu.unirio.br

Leonardo Souza
Universidade Federal do Estado do Rio de Janeiro (UNIRIO)
Rio de Janeiro, Rio de Janeiro, Brazil
leonardo.souza@edu.unirio.br

Sean Wolfgand Matsui Siqueira
Universidade Federal do Estado do Rio de Janeiro (UNIRIO)
Rio de Janeiro, Rio de Janeiro, Brazil
sean@uniriotec.br

Rodrigo Pereira dos Santos
Universidade Federal do Estado do Rio de Janeiro (UNIRIO)
Rio de Janeiro, Rio de Janeiro, Brazil
rps@uniriotec.br

ABSTRACT

Context: The proliferation of technologies based on Large Language Models (LLM) is reshaping various domains, also impacting on programming code creation and review. **Problem:** The decision-making process in adopting LLM in software development demands an understanding of associated challenges and diverse application possibilities. **Solution:** This study addresses the identified challenges linked to LLM utilization in programming code processes. It explores models, utilization strategies, challenges, and coping mechanisms, focusing on the perspectives of researchers in software development. **IS Theory:** Drawing on Task-Technology Fit (TTF) theory, the research examines the alignment between task characteristics in code generation and review, and LLM technology attributes to discern performance impacts and utilization patterns. **Method:** Employing the Systematic Mapping of the Literature method, the research analyzes 19 selected studies from digital databases—IEEE Digital Library, Compendex Engineering Village, and Scopus—out of 1,257 retrieved results. **Summary of Results:** The research reveals 23 models, 13 utilization strategies, 15 challenges, and 14 coping mechanisms associated with LLM in programming code processes, offering a comprehensive understanding of the application landscape. **Contributions to IS:** Contributing to the Information Systems (IS) field, This study provides valuable insights into the utilization of LLM in programming code generation and review. The identified models, strategies, challenges, and coping mechanisms offer practical guidance for decision-making processes related to LLM technology adoption. The research aims to support the IS community in effectively navigating the complexities of integrating large language models into the dynamic software development lifecycle.

CCS CONCEPTS

• **Information systems** → **Information systems applications**; • **Software and its engineering** → **Software development techniques**.

KEYWORDS

Code Generation; code completion; code auto-suggestion; automatic refactoring; natural language models, transformer architecture, neural network, LLM, systematic mapping study

ACM Reference Format:
Beatriz Ventorini Lins de Albuquerque, Antonio Fernando Souza da Cunha, Leonardo Souza, Sean Wolfgand Matsui Siqueira, and Rodrigo Pereira dos Santos. 2024. Generating and Reviewing Programming Codes with Large Language Models: A Systematic Mapping Study. In *XX Brazilian Symposium on Information Systems (SBSI '24), May 20–23, 2024, Juiz de Fora, Brazil.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3658271.3658342

1 INTRODUÇÃO

Tecnologias baseadas em Large Language Models (LLM) têm se expandido rapidamente, impulsionando transformações em diferentes áreas. LLM são modelos avançados de inteligência artificial (IA) que podem executar uma série de tarefas de processamento de linguagem natural (PLN), como ler, traduzir, classificar e resumir textos, gerar textos originais e responder perguntas. Esses modelos são baseados em algoritmos de aprendizado de máquina (ou *machine learning* - ML) treinados com uma quantidade massiva de dados textuais e utilizam cálculos probabilísticos para aprender com esses dados e prever palavras e frases que sejam coerentes com as instruções recebidas em linguagem natural [19, 25].

As organizações estão investindo cada vez mais para aproveitar o potencial dessas tecnologias, buscando otimizar suas operações,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBSI '24, May 20–23, 2024, Juiz de Fora, Brazil
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0996-8/24/05
https://doi.org/10.1145/3658271.3658342

economizar recursos e fortalecer a inovação. Dentre outras aplicações, os LLM podem ser utilizados para o desenvolvimento de tecnologias de IA conversacionais, como os assistentes virtuais, interfaces de voz e chatbots avançados [10].

Uma das áreas de aplicação de LLM é na geração e revisão automáticas de códigos de programação e há muitos desafios a serem explorados e compreendidos nesse contexto. A tomada de decisão sobre a aplicação dessa tecnologia nesse contexto requer a identificação e a compreensão das ferramentas, dos modelos de aplicação, dos desafios e dos mecanismos de enfrentamento (*coping mechanisms*) relacionados às diferentes possibilidades de utilização [10].

As organizações buscam constantemente soluções para melhorar sua eficiência e produtividade; nesse contexto, a automação da geração e revisão de códigos de programação é uma área de interesse significativa. Este estudo, ao investigar a utilização de tecnologias baseadas em LLM na geração e revisão de código de programação, pretende oferecer informações consolidadas e atualizadas para gestores e profissionais de tecnologia, possibilitando:

- Compreender melhor os modelos e ferramentas, os desafios e os mecanismos de enfrentamento associados à utilização de LLM na geração e revisão de código de programação, considerando as interações entre pessoas, processos e tecnologias;
- Apoiar o processo decisório sobre a adoção dessas tecnologias com objetivo de melhorar a eficiência de atividades de geração e revisão de código de programação;
- Incentivar e subsidiar reflexões e discussão sobre a maneira como o desenvolvimento de software é conduzido nas organizações.

O processo decisório sobre utilização de novas tecnologias demanda informações atualizadas e consolidadas sobre as possibilidades de utilização, os desafios relacionados a essa utilização e os mecanismos de enfrentamento para esses desafios. No caso de tecnologias emergentes, a falta de informações suficientes torna o processo decisório mais desafiador e arriscado. Considerando esse contexto, este estudo visa responder ao seguinte problema de pesquisa: como os LLM são utilizados na geração e revisão de códigos de programação, em termos de modelos e estratégias de utilização? Quais são os desafios relacionados à utilização dessas tecnologias e os mecanismos de enfrentamento para lidar com esses desafios?

Dessa forma, o objetivo deste estudo é investigar a utilização de LLM na geração e revisão de códigos de programação, com o propósito de caracterizar com respeito aos modelos, estratégias de utilização, desafios e mecanismos de enfrentamento *(coping mechanisms)*, do ponto de vista de pesquisadores, no contexto de pesquisas sobre desenvolvimento de software e sistemas de informação.

Para esse objetivo, foi utilizado o método de Mapeamento Sistemático da Literatura (MSL) nas bases de dados digitais IEEE Digital Library, Compendex Engineering Village e Scopus. Dos 1.257 estudos retornados, foram selecionados e analisados 19, trazendo como resultado 23 modelos, 13 estratégias de utilização, 15 desafios e 14 mecanismos de enfrentamento para esses desafios.

As perspectivas de análise são inspiradas no Modelo de Ajuste Tecnologia-Tarefa (ou em inglês *Technology-Task Fit (TTF) Theory*), partindo da premissa de que a tecnologia (no caso deste trabalho, LLM) tem maior probabilidade de ter um impacto positivo no desempenho dos usuários (programadores) e ser utilizada se as características da tecnologia (LLM) corresponderem às características da tarefa (geração e revisão de códigos de programação) que o usuário (programador) deverá realizar.

Além desta seção de introdução, este trabalho está organizado da seguinte forma: a Seção 2 resume os trabalhos relacionados; a Seção 3 apresenta a fundamentação teórica; a Seção 4 apresenta o planejamento e execução do estudo; a Seção 5 apresenta os resultados; a Seção 6 apresenta a discussão; a Seção 7 apresenta as ameaças à validade e as limitações deste estudo; e, por fim, a Conclusão apresenta as considerações finais.

2 TRABALHOS RELACIONADOS

Questões sobre geração e revisão automática de códigos de programação têm sido exploradas em trabalhos recentes (Tabela 1). Foram identificados diversos trabalhos que conduziram estudos secundários sobre temas de engenharia de software, e alguns deles investigaram questões especificamente associadas a geração e revisão automáticas de código de programação. Os trabalhos que conduziram estudos secundários mais diretamente relacionados ao problema de pesquisa deste estudo são apresentados na Tabela 1, sendo apresentados por ordem alfabética dos títulos dos artigos.

Tabela 1: Trabalhos relacionados

ID	Título do trabalho	Autores	Ano
T1	Code generation using machine learning: a systematic review [9]	E. Dehaerne et al.	2022
T2	Natural language generation and understanding of Big Code for AI-assisted programming: a review [33]	M.F. Wong et al.	2023
T3	Recent advances in intelligent source code generation: a survey on natural language based studies [34]	C. Yang; Y. Liu; C. Yin	2021
T4	Systematic literature review on solving competitive programming problem with artificial intelligence (AI) [1]	F. Alexander et al.	2022

T3 [34] conduziu um levantamento da literatura sobre os avanços no campo de Geração de Código Fonte baseada em Linguagem Natural (NLSCG), selecionando estudos publicados de 2015 a 2020; destacou a falta de estudos sistemáticos na área e indicou direções de pesquisa. T2 [9] conduziu uma revisão sistemática da literatura sobre geração de código usando métodos de aprendizado de máquina, incluindo estudos publicados de 2016 a 2021; identificou paradigmas de geração de código, modelos utilizados e aplicações desses modelos. T4 [1] conduziu uma revisão sistemática da literatura sobre o uso de IA na resolução de problemas de programação competitiva, com recorte dos estudos publicados de 2017 a 2022; discutiu a dificuldade de as IA pensarem como seres humanos e concluiu que as ferramentas de autocompletar código disponíveis não atendiam ao objetivo de resolver tarefas de programação competitiva com alto desempenho. T2 [33] conduziu uma revisão sistemática da literatura relacionada ao uso de técnicas de Processamento de Linguagem Natural (PLN), com ênfase em LLM na programação assistida por IA, com recorte dos estudos publicados entre 2019 e maio de 2023; as aplicações mencionadas incluem geração de código, conclusão de código, tradução de código e detecção de defeitos; esse trabalho também discute desafios e oportunidades associados à incorporação de PLN na geração de software. Dos trabalhos identificados,

T2 é o que aborda questões mais diretamente relacionadas ao problema de pesquisa deste estudo; a principal diferença é que T2 visou identificar aplicações gerais dos modelos investigados, enquanto este estudo buscou estudos primários que investigassem modelos, estratégias de utilização e desafios emergentes nesse campo, para identificar contribuições aplicadas e mais direcionadas a pesquisadores e profissionais no contexto de desenvolvimento de software. Este estudo também preenche uma lacuna ao se concentrar nas perspectivas e experiências de pesquisadores ativos no campo do desenvolvimento de software, o que possibilita uma visão mais aplicada das complexidades envolvidas na utilização de LLM para a geração e revisão de código de programação.

3 FUNDAMENTAÇÃO TEÓRICA

Os conceitos fundamentais relacionados ao objeto de estudo e que guiaram as questões estão organizados em três áreas temáticas:

- *Technology-Task Fit (TTF)*;
- *Large Language Models (LLM)*; e
- Geração e revisão automáticas de código de programação.

3.1 *Technology-Task Fit (TTF)*

Goodhue [12] parte da necessidade de identificar perspectivas teóricas que possam conectar de forma útil os sistemas subjacentes a seus impactos relevantes para implementar o *Technology-Task Fit (TTF)*[1], chamado em português de Modelo de Ajuste Tecnologia-Tarefa. Segundo o autor, a ênfase em produtividade e eficiência nos negócios sugere uma perspectiva que alie a tarefa à tecnologia.

Embora possa ser observado um foco no indivíduo na versão original [12] e um foco em grupos na versão de Zigurs and Buckland [35], Goodhue and Thompson [13] trazem uma visão mais geral, onde os elementos que direcionam o entendimento do ajuste são apenas as características da tarefa e as da tecnologia. Essa versão também reforça tanto os impactos no desempenho da tarefa, quanto a perspectiva de uso da tecnologia.

Este trabalho se inspira na TTF para entender como os LLM (a partir das características da tecnologia) são utilizados (perspectiva de uso no modelo TTF) para geração e revisão de código de programação (a partir das características da tarefa). Também são levantadas as questões relacionadas aos impactos no desempenho do programador (perspectiva de impacto no desempenho do TTF) a partir dos desafios relacionados ao uso de LLM no contexto da geração e revisão de código de programação, e os mecanismos de enfrentamento utilizados para lidar com esses desafios.

3.2 *Large Language Models (LLM)*

Inteligência artificial (IA) pode ser definida como um conjunto de modelos (de busca, representação de conhecimento, decisão, processamento de dados, tratamento de incertezas, aprendizado e outras técnicas) que, isoladamente ou agrupados, são desenvolvidos para resolver problemas de natureza complexa que seriam usualmente mais bem solucionados por seres humanos [26]. Há vários tipos de modelos de IA; dentre eles, os modelos de IA generativa, que são capazes de gerar conteúdo original a partir do treinamento com grandes volumes de dados, sem depender de regras ou conhecimentos pré-codificados. Os modelos de IA generativa originalmente foram desenvolvidos para trabalhar com dados matemáticos e estatísticos, mas a ascensão de aprendizagem profunda (*deep learning*) possibilitou ampliar sua aplicação a imagens, linguagem natural, música e a outros tipos de dados complexos [19]. Os LLM, que são os modelos de maior interesse para este estudo, consistem em modelos de IA generativa específicos para produção de conteúdo coerente na forma textual; são treinados com massivas bases de dados textuais e utilizam cálculos probabilísticos para prever palavras e frases que sejam coerentes com as instruções recebidas dos usuários em linguagem natural [19, 25].

3.3 Geração e revisão automáticas de código de programação

Uma das áreas de aplicação de LLM é na geração e revisão automáticas de códigos de programação, e essa aplicação vem sendo explorada para investigar a eficácia dos modelos na geração de códigos que possam otimizar o trabalho de desenvolvedores. Pesquisas sobre recursos para geração automática de código não são recentes; muitas ferramentas de desenvolvimento já incorporam recursos para tarefas como identificação e sinalização de erros na sintaxe do código, sugestão de trechos de código conforme o usuário digita uma linha de código na ferramenta (*autocomplete*) e refatoração de códigos seguindo boas práticas de mercado [9]. No entanto, a utilização de LLM pode ampliar significativamente essa otimização das tarefas de desenvolvimento de código e possibilitar a automação de tarefas mais complexas [9] [34].

Embora seja possível gerar código de programação utilizando linguagem natural por meio de *prompts* em *chatbots* baseados em LLM, as tarefas de desenvolvimento de código envolvem regras estritas de sintaxe e de lógica e pode ser desafiador conseguir gerar códigos com alta qualidade em *chatbots* não especializados em geração de código de programação [11].

O avanço em pesquisas sobre LLM especializados para geração de código possibilitou o uso mais efetivo de técnicas de síntese automática de programas e, através dessa técnica, os modelos conseguem gerar novos códigos que podem ser utilizados na educação, no desenvolvimento de assistentes e na descoberta de novos algoritmos; no entanto, é importante enfatizar que o avanço nos LLM está diretamente ligado à evolução de uma arquitetura de *Deep Learning* específica conhecida como *Transformer*, que aumenta o desempenho dos modelos na síntese de programas [8].

4 MÉTODO DE PESQUISA

O método utilizado foi o Mapeamento Sistemático da Literatura (MSL), que consiste em um estudo secundário com o objetivo de caracterizar o estado da arte sobre determinado tema. Envolve a classificação dos tópicos estudados em uma área de pesquisa, seguindo procedimentos metodológicos sistemáticos para diminuir as ameaças à validade dos resultados. São levantados os tópicos mais estudados, lacunas de pesquisa, abordagens e tipos de pesquisa identificados, resultados obtidos, possíveis restrições e desafios [15].

[1] https://is.theorizeit.org/wiki/Task-technology_fit

4.1 Planejamento

Nesta etapa, definiu-se o protocolo de pesquisa: a) objetivo; b) questões de pesquisa; c) estratégia de busca; d) critérios de exclusão e inclusão; e) procedimentos de seleção; e f) estratégias para extração e síntese dos dados.

4.2 Objetivo

O objetivo do estudo foi definido a partir do paradigma GQM [4]: analisar a utilização de LLM na geração e revisão de códigos de programação, com o propósito de caracterizar com respeito aos modelos, estratégias de utilização, desafios e mecanismos de enfrentamento (*coping mechanisms*), do ponto de vista de pesquisadores, no contexto de pesquisas sobre desenvolvimento de software e sistemas de informação.

4.3 Questões de Pesquisa

Para atingir o objetivo da pesquisa, foram formuladas duas questões de pesquisa (Tabela 2). Essas questões foram propostas com o intuito de identificar como os modelos de LLM têm sido utilizados na geração e revisão de códigos de programação. Para isso, é necessário compreender melhor os desafios associados à utilização de LLM na geração e revisão de códigos de programação, e também identificar e compreender os mecanismos de enfrentamento (*coping mechanisms*) que têm sido utilizados ou propostos para lidar com esses desafios.

Tabela 2: Questões de pesquisa

ID	Questão de pesquisa	Meta da questão
QP1	Como os LLM são utilizados para geração e revisão de códigos de programação?	Identificar os modelos e as estratégias de utilização de LLM na geração e revisão de código.
QP2	Quais são os desafios relacionados à utilização de LLM para geração e revisão de códigos de programação, e quais mecanismos de enfrentamento *coping mechanisms* são utilizados ou propostos para lidar com esses desafios?	Identificar os desafios e os mecanismos de enfrentamento relacionados à utilização de LLM na geração e revisão de código.

4.4 Estratégia de busca

As buscas por estudos primários para responder às questões de pesquisa (QP) apresentadas na tabela 2 foram realizadas durante o mês de setembro de 2023 nas seguintes bibliotecas digitais:

- IEEE Digital Library (http://ieeexplore.ieee.org);
- Compendex Engineering Village (https://www.engineeringvillage.com/);
- Scopus (https://www.scopus.com/).

Para padronizar essas buscas, foi definida uma *string* de busca a partir de duas dimensões do modelo PICOC [23]: população (P) e intervenção (I). A comparação (C) não se aplica a mapeamentos, enquanto resultado (O) e contexto (C) limitavam excessivamente os resultados. Essa *string* foi testada em diferentes versões nas bibliotecas digitais, até que os resultados retornados foram considerados consistentes.

A versão final da *string* de busca foi a seguinte: ("code Generation" OR "code completion" OR "code auto-suggestion" OR "program-aided-language" OR "automatic refactoring") AND ("AI chatbot" OR "conversational AI" OR "generative AI" OR "natural language" OR"transformer architecture" OR "neural network" OR LLM OR large language model OR chatGPT OR copilot).

Para realização das buscas nas diferentes bibliotecas digitais, a sintaxe da *string* foi adaptada, mantendo a mesma estrutura de sua forma padrão. A busca foi realizada nos campos título, resumo *(asbtract)* e palavras-chave *(keywords)* de cada biblioteca. Os resultados foram compilados em uma planilha unificada[2] para, em seguida, serem aplicados os critérios de exclusão e inclusão de estudos.

4.5 Critérios de exclusão e inclusão

Os estudos que retornaram com a utilização da *string* de busca nas bibliotecas digitais mencionadas foram selecionados por meio da aplicação de seis critérios de exclusão (CE) e de um critério de inclusão (CI), apresentados a seguir:

- **CE1:** Estudo duplicado.
- **CE2:** Estudo se refere a tutorial, mesa redonda, palestra, prefácio, livro, *preprint* ou editorial.
- **CE3:** Estudo não escrito em inglês.
- **CE4:** Estudo secundário.
- **CE5:** Estudo fora do escopo da pesquisa.
- **CE6:** Estudo não acessível em texto completo.
- **CI1:** Estudo investiga modelos, estratégias de utilização, desafios e/ou mecanismos de enfrentamento relacionados à utilização de LLM na geração e/ou revisão de códigos de programação.

Os estudos passaram sequencialmente pela análise dos CE e, caso se enquadrassem em algum deles, foram excluídos, sem serem avaliados pelos critérios seguintes. Apenas os estudos que não se enquadraram em nenhum dos CE foram analisados quanto ao CI.

4.6 Procedimentos de Seleção

A Figura 1 apresenta uma visão geral dos resultados de cada etapa do processo de seleção dos estudos.

O processo de seleção foi conduzido por três dos cinco autores deste trabalho; cada pesquisador aplicou individualmente o primeiro filtro de seleção e, no final, as planilhas com os resultados de cada pesquisador foram comparadas; os casos unânimes foram mantidos e, nos casos em que houve algum impasse, os pesquisadores revisaram suas análises até chegarem a um consenso. Foram identificados inicialmente 1.257 estudos nas três bases digitais consultadas; após aplicação do filtro referente aos estudos duplicados (CE1), restaram 717 estudos; após a aplicação dos outros critérios de exclusão (CE2, CE3, CE4, CE5 e CE6), utilizando como referência o título, o resumo e as palavras-chave dos estudos, 132 foram selecionados. Em seguida, os estudos selecionados foram organizados em três subconjuntos para a análise do critério de inclusão (CI); cada subconjunto foi analisado por dois pesquisadores, em duas rodadas individuais, alternando os pesquisadores para confirmação dos estudos que seriam incluídos na lista final. Para a análise do CI, foi analisado o texto completo dos estudos selecionados nas etapas

[2]A planilha com os metadados dos estudos identificados na busca estão disponíveis no endereço: https://doi.org/10.5281/zenodo.10343613

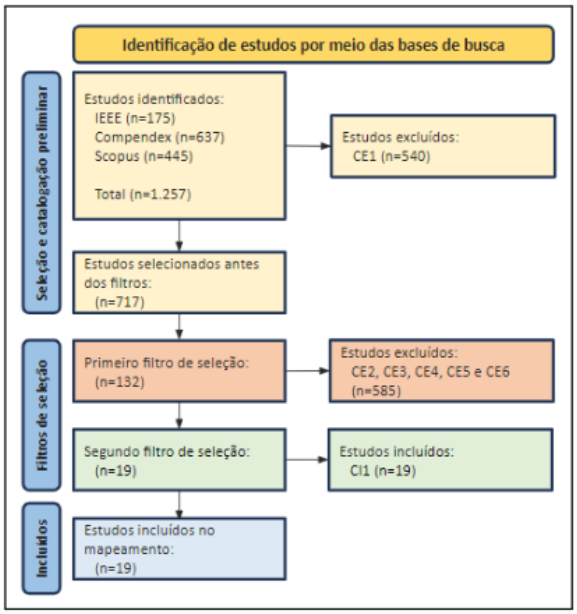

Figura 1: Processo de busca e seleção dos estudos com base na abordagem PRISMA [21]

anteriores. Nessa última etapa, foram selecionados 19 estudos; a lista final foi revisada pelos três pesquisadores em conjunto.

4.7 Extração e síntese dos dados

O MSL foi executado em setembro de 2023, seguindo os procedimentos e os critérios definidos no protocolo[3].

5 RESULTADOS

A Figura 2 apresenta o número de estudos por ano de publicação e a Figura 3 apresenta o número de estudos por país de afiliação dos pesquisadores (universidade, empresa, centro de pesquisa). Alguns estudos envolveram pesquisadores de mais de um país.

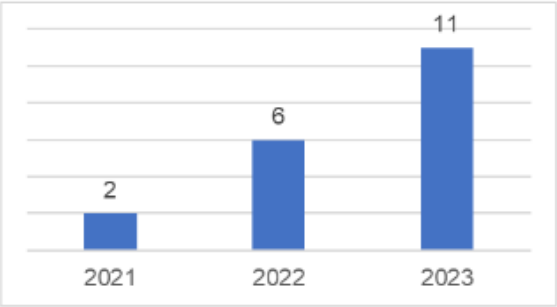

Figura 2: Número de estudos por ano de publicação

[3]As planilhas com os metadados dos estudos incluídos, os dados extraídos e os resultados consolidados estão disponíveis no endereço: https://doi.org/10.5281/zenodo.10343613

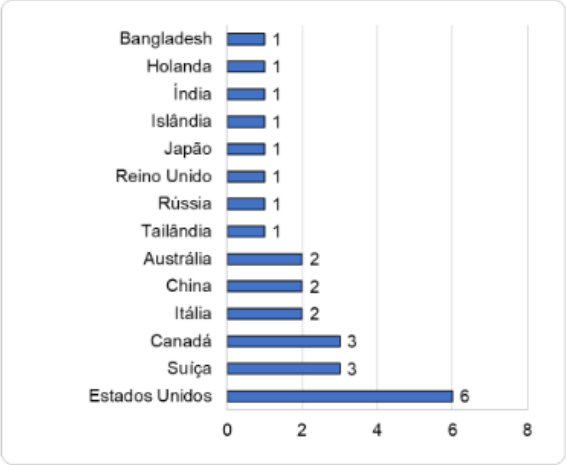

Figura 3: Número de participações por país

A Tabela 3 lista os tipos de estudos identificados e indica os estudos que responderam a cada uma das QPs. Quanto ao tipo de estudo, 17 estudos relataram ter conduzido estudos experimentais; 2 estudos (E04 e E05) conduziram estudos de caso.

Quanto às respostas às QPs, 16 estudos continham informações suficientes para responder à QP1 e 10 estudos à QP2; 3 estudos responderam parcialmente à QP1; 8 responderam parcialmente à QP2; 1 estudo não respondeu à QP2.

Tabela 3: Estudos que respondem as QPs

ID	Tipo de estudo	Responde QP1?	Responde QP2?
E01	Experimento	Sim	Parcialmente
E02	Experimento	Parcialmente	Parcialmente
E03	Experimento	Sim	Parcialmente
E04	Estudo de caso	Sim	Sim
E05	Estudo de caso	Sim	Sim
E06	Experimento	Sim	Sim
E07	Experimento	Sim	Parcialmente
E08	Experimento	Sim	Parcialmente
E09	Experimento	Sim	Sim
E10	Experimento	Sim	Parcialmente
E11	Experimento	Sim	Sim
E12	Experimento	Sim	Sim
E13	Experimento	Sim	Parcialmente
E14	Experimento	Sim	Sim
E15	Experimento	Sim	Parcialmente
E16	Experimento	Sim	Sim
E17	Experimento	Parcialmente	Sim
E18	Experimento	Sim	Sim
E19	Experimento	Parcialmente	Não

Quanto à QP1, todos os 19 estudos investigaram um ou mais modelos utilizados na geração ou revisão de códigos de programação. No entanto, 3 estudos não especificaram quais estratégias de utilização foram consideradas para utilização dos modelos investigados.

Quanto à QP2, 18 estudos mencionaram ou investigaram desafios para a utilização de LLM na geração e revisão de código de programação; desses, 10 investigaram, testaram ou propuseram mecanismos de enfrentamento para esses desafios.

A Tabela 4 apresenta um resumo da extração de dados dos 19 estudos incluídos, organizados em ordem alfabética pelo título do estudo.

Tabela 4: Lista final dos estudos incluídos

ID	Título do estudo	Autores	Países de afiliação	Ano
E01	An Empirical Evaluation of Competitive Programming AI: A Case Study of AlphaCode. [17]	S. Lertbanjongngam et al.	Tailândia; Japão	2022
E02	An Empirical Study of Code Smells in Transformer-based Code Generation Techniques. [27]	M. L. Siddiq et al.	Estados Unidos; Bangladesh	2022
E03	Automated Source Code Generation and Auto-Completion Using Deep Learning: Comparing and Discussing Current Language Model-Related Approaches. [7]	J. Cruz-Benito et al.	Estados Unidos	2021
E04	Chat2Code: A Chatbot for Model Specification and Code Generation, The Case of Smart Contracts. [24]	I. Qasse et al.	Islândia; Suíça; Canadá	2023
E05	Competition-level code generation with AlphaCode. [18]	Y. Li et al.	Reino Unido	2022
E06	Don't Complete It! Preventing Unhelpful Code Completion for Productive and Sustainable Neural Code Completion Systems. [29]	Z. Sun et al.	Austrália; China	2023
E07	Enriching Source Code with Contextual Data for Code Completion Models: An Empirical Study. [31]	T. Dam; M. Izadi; A. Deursen	Holanda	2023
E08	Examining Zero-Shot Vulnerability Repair with Large Language Models. [22]	H. Pearce et al.	Estados Unidos; Canadá	2023
E09	GitHub Copilot AI pair programmer: Asset or Liability? [8]	A.M. Dakhel et al.	Canadá	2023
E10	Investigating Code Generation Performance of ChatGPT with Crowdsourcing Social Data. [11]	Y. Feng et al.	Estados Unidos	2023
E11	MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. [5]	F. Cassano et al.	Estados Unidos	2023
E12	NLI-GSC: A Natural Language Interface for Generating SourceCode. [2]	A. A. R. H. Ansari; D. R. Vora	Índia	2022
E13	On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot. [20]	A. Mastropaolo et al.	Suíça; Itália	2023
E14	Test-based and metric-based evaluation of code generation models for practical question answering. [16]	S. Kovalchuk et al.	Rússia	2023
E15	To What Extent do Deep Learning-based Code Recommenders Generate Predictions by Cloning Code from the Training Set? [6]	M. Ciniselli; L. Pascarella; G. Bavota	Suíça	2022
E16	Towards More Effective AI-Assisted Programming: A Systematic Design Exploration to Improve Visual Studio IntelliCode's User Experience. [30]	P. Vaithilingam et al.	Estados Unidos e outros	2023
E17	What do pre-trained code models know about code? [14]	A. Karmakar; R. Robbes	Itália	2021
E18	What Do They Capture? - A Structural Analysis of Pre-Trained Language Models for Source Code. [32]	Y. Wan et al.	China; Austrália	2022
E19	Zero-shot Prompting for Code Complexity Prediction Using GitHub Copilot. [28]	M. L. Siddiq et al.	Estados Unidos; Bangladesh	2023

Nas subseções seguintes serão apresentados os resultados obtidos para cada uma das QPs.

5.1 Modelos e estratégias de utilização (QP1)

A Tabela 5 apresenta os modelos que foram investigados nos estudos e a lista dos estudos que citaram cada modelo.

Foram identificados 23 modelos e 13 diferentes estratégias de utilização. Os modelos mais citados foram GitHub Copilot (6 estudos) e BERT (3 estudos). 8 modelos foram citados em 2 estudos (Alphacode, CodeBERT, CodeBERTa, CodeGen, GraphCodeBERT, InCoder, LLM não especificado e OpenAI Codex) e os modelos restantes foram citados em apenas um estudo cada.

Quanto às estratégias de utilização dos modelos, os estudos investigaram diferentes abordagens relacionadas ao uso de LLM na geração e revisão de código de programação, abrangendo desde a comparação de desempenho de modelos até a melhoria da eficiência e a aplicação prática desses modelos. Alguns estudos focaram em linguagens de programação específicas, outros abordaram as questões de um ponto de vista mais abrangente:

- Comparação empírica de código gerado por LLM e por humanos: alguns estudos realizaram comparações entre código gerado por LLM e código desenvolvido por humanos. E01 [17] apresentou uma comparação empírica entre código gerado pelo AlphaCode e código desenvolvido por humanos; E09 [8] investigou se Copilot seria capaz de gerar códigos eficientes para resolver algoritmos fundamentais da computação, superando soluções humanas.

Tabela 5: Respostas à QP1: Modelos investigados

Modelos	Estudos
GitHub Copilot	E02, E06, E09, E13, E15, E19
BERT	E03, E17, E18
Alphacode	E01, E05
CodeBERT, CodeBERTa	E17, E18
GraphCodeBERT	E17, E18
CodeGen	E11, E14
InCoder	E07, E11
LLM	E04, E08
OpenAI Codex	E08, E11
Chatbot	E04
ChatGPT	E10
Code Generation System	E01
CodeGPT	E07
DL Models	E15
GPT-2	E03
GPT-Code-Clippy	E02
GPT-Neo	E14
LCM	E06
Microsoft Visual Studio Intellicode	E16
MultiPL-E	E11
NLI-GSI	E12
UniXcoder	E07

- Comparação empírica do desempenho de modelos: E11 [5] testou o desempenho de três modelos de geração de código (OpenAI Codex, InCoder, CodeGen) na criação de código a partir de instruções em linguagem natural, usando comandos em *prompts*; E10 [11] e E14 [16] realizaram experimentos para comparar o desempenho de modelos de geração de código por meio de métricas de PLN.
- Detecção de problemas de código e correção de vulnerabilidades: E02 [27] se concentrou na investigação da existência de "*code smells*" nos dados de treinamento dos modelos e na possível propagação desses problemas para o código gerado; E08 [22] investigou a utilização de LLM para reparo de vulnerabilidades.
- Comparação de arquiteturas de Redes Neurais: E03 [7] comparou diferentes arquiteturas de redes neurais usando aprendizado por transferência e técnicas de tokenização para a construção de modelos de linguagem voltados especificamente para geração de código em Python.
- Engenharia Dirigida por Modelos (MDE): E04 [24] explorou a geração automática de código em *frameworks* de MDE, com foco na autogeração de código de contratos inteligentes com base em sintaxe conversacional.
- Propostas de novos modelos ou ferramentas: E05 [18] apresentou o sistema AlphaCode, projetado para gerar código de nível competitivo; E11 [5] apresentou o MultiPL-E, um sistema desenvolvido pelos autores para apoiar a avaliação de desempenho dos modelos de geração de código em diferentes linguagens de programação, traduzindo *benchmarks* de geração de código de uma linguagem para outras; E12 [2] apresentou o NLI-GSC, uma ferramenta independente de linguagem para escrever código de programação, que utiliza PLN para converter linguagem natural em pseudocódigo independente de linguagem e, em seguida, converte o pseudocódigo em código de uma linguagem específica.
- Abordagens para melhorar a eficiência na geração de código: E06 [29] destacou que grande parte dos códigos gerados por LLM não é útil e investigou abordagens para melhorar a eficiência, como a rejeição precoce de códigos de baixo retorno.
- Adição de contexto para melhorar o desempenho: E07 [31] analisou a possibilidade de melhorar o desempenho dos modelos incluindo mais dados de contexto no treinamento, como comentários e informações sobre tipos de variáveis nos códigos.
- Redes sociais como fonte de dados: E10 [11] explorou o desempenho dos modelos com dados provenientes de redes sociais, como Reddit e Twitter.
- Geração automática de funções: E13 [20] investigou a geração automática de funções completas a partir de descrições em linguagem natural.
- Clonagem de código: E15 [6] investigou até que ponto os modelos de aprendizagem profunda tendem a clonar código de seus conjuntos de treinamento ao realizar o preenchimento automático de código.
- Recomendações de código em tempo real: E16 [30] investigou em profundidade o Microsoft Visual Studio Intellicode e suas funcionalidades de recomendações de código em tempo real para inserção ou modificação de trechos de código.
- Análise estrutural dos modelos: E17 [14] investigou desafios relacionados à capacidade dos modelos pré-treinados de capturar, de forma eficaz, características do código-fonte; E18 [32] realizou uma análise estrutural dos modelos para geração de código e as possíveis aplicações desses modelos, como preenchimento, pesquisa e resumo de código; E19 [28] investigou a capacidade de o GitHub Copilot prever a complexidade do código usando *zero-shot prompting*.

5.2 Desafios e mecanismos de enfrentamento (QP2)

Quanto aos desafios para a utilização de LLM na geração e revisão de código, os estudos investigaram e/ou mencionaram tópicos bastante diversificados, desde desafios relacionados à qualidade do código gerado até questões de eficiência e interpretabilidade. A seguir, a consolidação dos 15 desafios identificados nos estudos:

- Desempenho do código gerado: E01 [17] e E09 [8] destacaram que, em diferentes aspectos, o desempenho de códigos gerados pelos modelos estudados pode ser inferior ao desempenho de códigos produzidos por humanos, com base em métricas como tempo de execução e uso de memória.
- Qualidade do código gerado: E02 [27] destacou que padrões de código ruins podem vazar para a saída dos modelos geradores de código, incluindo problemas de código de segurança, e enfatizou a importância de verificar a qualidade do código gerado; E09 [8] identificou em testes que a qualidade do código gerado por um modelo ficou abaixo da qualidade do código desenvolvido por humanos; E14 [16] apontou que o uso de métricas em PLN não é suficiente para aferir a qualidade do código produzido.
- Problemas nos dados de treinamento: E02 [27] investigou problemas nos conjuntos de dados de treinamento usados pelos modelos, que podem sofrer problemas de qualidade, demandando pesquisas adicionais sobre técnicas de pré-processamento para removê-los ou repará-los; E05 [18] e E12 [2] identificaram desafios na obtenção de conjuntos de dados adequados para treinamento e avaliação de modelos.
- Comparação de arquiteturas de aprendizado profundo: E03 [7] destacou que faltam trabalhos empíricos que comparem diferentes arquiteturas de aprendizado profundo para criar e usar modelos de linguagem baseados em código de programação.
- Especificação de requisitos: E04 [24] identificou a necessidade de que especialistas do domínio aprendam a especificar requisitos de aplicação usando sintaxe concreta.
- Custo computacional e eficiência: E06 [29] investigou o alto custo computacional associado a modelos de *deep learning* e infraestrutura, particularmente para oferecer *autocomplete* com menor probabilidade de rejeição.
- Contexto adicional na geração de código: E07 [31] destacou a importância de se investigar o desempenho de modelos treinados com códigos que contêm informações de contexto mais completas e detalhadas.

- Reparo de falhas de segurança: E08 [22] identificou a necessidade de se investigar se modelos de LLM podem ser úteis no reparo de falhas de segurança nos códigos.
- Avaliação de modelos: E11 [5] investigou o desafio da falta de evidências quantitativas para avaliar o desempenho de modelos de geração de código em linguagens de programação que não sejam Python; E10 [11] identificou a falta de estudos de larga escala sobre o desempenho de LLM na geração de código.
- Limitação na conversão de linguagem natural em código: E12 [2] investigou as limitações da conversão de linguagem natural em código-fonte pelos modelos.
- Robustez na geração de código: E13 [20] identificou problemas na robustez dos geradores de código baseados em aprendizado profundo, incluindo a geração de códigos diferentes para descrições de linguagem natural semanticamente equivalentes.
- Originalidade, plágio e licenciamento: E15 identificou desafios envolvendo a determinação da originalidade do código gerado e possíveis implicações de licenciamento e clonagem de código.
- Subutilização de recomendações de alterações: E16 [30] investigou por que, na ferramenta Microsoft Visual Studio Intellicode, a funcionalidade de recomendação para alterar código é subutilizada em comparação com a funcionalidade de autocompletar.
- Captura de características do código: E17 [14] investigou desafios relacionados à capacidade dos modelos pré-treinados de capturar características do código de forma eficaz.
- Interpretabilidade de modelos de linguagem: E18 [32] investigou desafios relacionados à interpretabilidade e ao entendimento dos modelos em relação ao código-fonte.

Quanto aos mecanismos de enfrentamento para os desafios investigados, apenas 10 estudos apresentam claramente algumas recomendações ou direcionamentos. Os mecanismos apresentados abrangem desde recomendações para melhoria da qualidade do código até modificações na interface para os usuários. A seguir, a consolidação dos 14 mecanismos de enfrentamento identificados:

- Uso de Analisadores Estáticos e *Linters*: para enfrentar desafios relacionados à qualidade do código gerado, E02 [27] sugere o uso de analisadores estáticos e/ou *linters* para verificar a qualidade das sugestões de geração de código.
- Interfaces conversacionais e chatbots: a utilização de interfaces conversacionais, como *chatbots*, é discutida por E04 [24] como um mecanismo para melhorar o processo de aprendizado e de especificação interativa de requisitos de aplicações.
- Estratégias dos desenvolvedores: E05 [18] investiga estratégias adotadas pelos desenvolvedores, como o uso de conjuntos de dados extensos e limpos, para enfrentar o desafio de obtenção de datasets para treinamento e avaliação dos modelos.
- Estimador baseado em *Transformer*: E06 [29]propõe o uso de um estimador baseado em *Transformer* para verificar a viabilidade do mecanismo de rejeição de código, economizando recursos computacionais e melhorando a qualidade do recurso de autocompletar.
- *Workflow* de teste do modelo: para enfrentar os desafios relacionados a desempenho e qualidade do código gerado pelo Github Copilot, E09 [8] propõe um *workflow* de teste do modelo, avaliando sua capacidade de recomendação de código e de resolução de problemas, com testes baseados em livros de programação e tarefas de estudantes de programação.
- Ferramentas desenvolvidas pelos autores: alguns estudos apresentam ferramentas e modelos autorais para lidar com os desafios investigados; E11 [5] apresenta o MultiPL-E para enfrentar o desafio de avaliação dos modelos; E12 [2] apresenta o NLI-GSC para o desafio de conversão de linguagem natural em código-fonte.
- Criação de conjuntos de dados personalizados: E12 [2] propõe a criação de conjuntos de dados personalizados, preenchidos com palavras-chave predefinidas, para lidar com desafios relacionados à qualidade dos dados de treinamento.
- Ferramentas independentes de linguagem: para enfrentar o desafio da falta de flexibilidade na geração de código em diferentes linguagens de programação, E12 [2] propõe o uso de ferramentas independentes de linguagem que trabalhem em duas etapas: convertam linguagem natural em pseudocódigo e depois gerem o código na linguagem definida.
- Melhorias na robustez dos geradores de código: E13 [20] não apresenta uma solução para os desafios relacionados à semântica e exatidão dos códigos gerados, mas indica a necessidade de se testar e melhorar a robustez dos geradores de código baseados em aprendizado profundo.
- Avaliação multifacetada: E14 [16] recomenda a combinação de diversas avaliações, incluindo métricas de PLN, testes de códigos e avaliações automáticas para lidar com o desafio de aferir a qualidade dos códigos gerados.
- Princípios de design para interface: para lidar com a subutilização de ferramentas de recomendação de alterações no código, E16 [30] propõe um conjunto de princípios de design de interface, que inclui a comparação lado a lado de código original e modificado, a reutilização de interfaces existentes e o fornecimento proativo de sugestões.
- Tarefas de sondagem para avaliação de modelos: um estudo recomenda o uso de tarefas de sondagem para avaliar modelos de código pré-treinados em várias dimensões, identificando pontos fortes e fracos dos modelos.
- Incorporação da estrutura de sintaxe: E17 [14] recomenda a incorporação da estrutura da sintaxe do código no processo de pré-treinamento para obter melhores representações do código no modelo.
- Necessidade de mais pesquisas e discussões: 8 estudos não propõem mecanismos de enfrentamento para os desafios que identificaram, mas ressaltam a importância de mais pesquisas e discussões sobre a utilização de LLM na geração e revisão de código de programação.

6 DISCUSSÃO

Neste estudo, realizamos um mapeamento sistemático da literatura sobre a utilização de *Large Language Models* (LLM) na geração e revisão de códigos de programação. Os resultados revelaram uma ampla gama de LLM sendo empregados em diferentes estratégias de

geração e revisão de código. Alguns modelos, como o GitHub Copilot e o BERT, emergiram como os mais citados neste mapeamento. A diversidade de estratégias de utilização demonstra o potencial dessas tecnologias em diversos cenários de desenvolvimento de software. Uma das questões levantadas pelos estudos mapeados foi a capacidade desses modelos de gerar código que, em alguns casos, rivaliza com o código produzido por humanos. Isso pode ser particularmente promissor para melhorar a eficiência e a produtividade no desenvolvimento de software. Além disso, os estudos mapeados também realizaram experimentos de comparação empírica do desempenho de modelos, detecção de problemas de código e correção de vulnerabilidades.

No entanto, também foram identificados nos estudos diversos desafios relacionados à utilização de LLM na geração de código. A qualidade do código gerado, por exemplo, foi uma preocupação significativa, com a possibilidade de padrões de código ruins serem replicados pelos modelos. A avaliação da qualidade do código gerado também apresenta desafios, uma vez que métricas tradicionais de PLN nem sempre são adequadas para medir a qualidade. Outros desafios identificados incluem a obtenção de conjuntos adequados de dados de treinamento e a robustez dos geradores de código baseados em aprendizado profundo, que podem gerar códigos diferentes para descrições de linguagem natural semanticamente equivalentes. Além disso, questões éticas e legais, como licenciamento e clonagem de código, também foram identificadas.

Para enfrentar esses desafios, alguns dos estudos mapeados propuseram uma série de mecanismos de enfrentamento. Esses mecanismos variam desde o uso de analisadores estáticos e *linters* para melhorar a qualidade do código até a criação de conjuntos de dados personalizados para treinamento. Estratégias relacionadas ao tratamento dos dados de treinamento também foram sugeridas por diferentes estudos. Além disso, melhorias na robustez dos geradores de código e o uso de princípios de design de interface foram recomendados para aprimorar a usabilidade dessas tecnologias. Embora os estudos tenham apresentado várias recomendações e propostas, quase a metade dos estudos mapeados não apresentou nenhum mecanismo de enfrentamento para os desafios identificados. Alguns temas são emergentes e alguns estudos mencionam que esses desafios permanecem sem solução. Isso torna evidente a necessidade de pesquisas adicionais e discussões contínuas sobre o uso de LLM na geração e revisão de código de programação.

Este estudo fornece uma visão abrangente do atual cenário de utilização de LLM na geração e revisão de código de programação; destaca os modelos utilizados, as estratégias de utilização, os desafios associados a essa tecnologia e os mecanismos de enfrentamento para esses desafios. Espera-se que esses resultados sejam úteis para pesquisadores e profissionais que buscam explorar o potencial dos LLM na melhoria dos processos de desenvolvimento de software.

7 AMEAÇAS À VALIDADE E LIMITAÇÕES DO ESTUDO

Buscamos minimizar as ameaças à validade de construto, que se refere à capacidade de o estudo responder o problema de pesquisa, por meio de uma busca em bibliotecas digitais com uma ampla gama de estudos e pela utilização de uma *string* de busca construída a partir do modelo PICOC, com termos buscados em estudos relevantes sobre temas correlatos. Quanto à validade de confiabilidade, que se refere à potencial replicabilidade, buscamos minimizar as ameaças pela disponibilização pública das planilhas com os *datasets* que foram utilizados na seleção dos estudos e na extração dos dados. Para minimizar o viés do pesquisador, cada estudo identificado foi avaliado por ao menos dois pesquisadores de forma independente. Quanto às limitações do estudo: utilizou apenas três bibliotecas digitais que, apesar de abrangerem uma ampla gama de estudos, possivelmente não contemplam todo o conjunto existente de estudos sobre o tema; e na seleção dos estudos, o texto completo só foi avaliado na etapa de aplicação do critério de inclusão, enquanto para os critérios de exclusão foram analisados apenas o título, o resumo e as palavras-chave, o que pode ter provocado a exclusão de estudos que seriam adequados, caso tenha ocorrido algum tipo de inadequação nos meta-dados nas bibliotecas digitais e/ou o título dos estudos não seja representativo do tema pesquisado.

8 CONSIDERAÇÕES FINAIS

Contribuindo para o campo de Sistemas de Informação (SI), este estudo se baseia e fundamenta no modelo de ajuste tecnologia-tarefa (*Task-Technology Fit (TTF) Theory*) ao observar a relação da utilização de LLM (características da tecnologia) na geração e revisão de código de programação (características da tarefa). Deste modo, esta pesquisa mapeou estudos sobre os modelos, estratégias de utilização, desafios e mecanismos de enfrentamento associados à utilização de LLM na geração e revisão de códigos de programação. Foram identificados diversos modelos e estratégias de aplicação, demonstrando o amplo potencial de LLM para melhorar a eficiência e produtividade no desenvolvimento de software. Os resultados oferecem orientação prática para processos de tomada de decisão relacionados à adoção dessa tecnologia nesse domínio. No entanto, também foi indetificado um conjunto significativo de desafios, relacionados a questões como qualidade do código gerado, confiabilidade e robustez dos modelos, métricas, segurança de código e questões éticas e legais. Os mecanismos de enfrentamento propostos pelos estudos mapeados oferecem abordagens promissoras para lidar com esses desafios. Assim, a pesquisa visa apoiar a comunidade de SI na navegação eficaz pelas complexidades da integração de grandes modelos de linguagem no ciclo de vida dinâmico de desenvolvimento de software. Vale destacar a contribuição deste trabalho para os Grandes Desafios de Pesquisa em Sistemas de Informação (GranDSI-Br 2016-2026) [3], particularmente a Sistemas de Informação e os Desafios do Mundo Aberto. Ao promover a geração e revisão de código de programação, também espera-se contribuir com novas iniciativas no mundo aberto. É importante, entretanto, ressaltar que a pesquisa nesse campo está em constante evolução, e muitas questões permanecem sem solução definitiva. Portanto, esta análise destaca a necessidade contínua de pesquisas adicionais e discussões colaborativas para explorar plenamente o potencial de LLM na geração e revisão de código e para abordar as complexidades inerentes a essa tecnologia em constante desenvolvimento.

AGRADECIMENTOS

Os autores agradecem ao CNPq, FAPERJ, CAPES e UNIRIO pelo suporte. O estudo foi parcialmente financiado pelo CNPq (Proc.

316510/ 2023-8 e 305436/2021-0), pela FAPERJ (Proc. 211.583/2019 e E-26/210.792/2024), pela CAPES (Programa de Apoio à Disseminação de Informação Científica e Tecnológica - PADICT e Portal de Periódicos da CAPES) e pela UNIRIO.

REFERÊNCIAS

[1] Francis Alexander, Edwin Ario Abdiwijaya, Felix Pherry, Alexander Agung Santoso Gunawan, and Anderies. 2022. Systematic Literature Review on Solving Competitive Programming Problem with Artificial Intelligence (AI). In *2022 1st International Conference on Software Engineering and Information Technology (ICoSEIT)*. 85–90. https://doi.org/10.1109/ICoSEIT55604.2022.10029949

[2] Aaqib Ahmed R. H. Ansari and Deepali R. Vora. 2022. NLI-GSC: A Natural Language Interface for Generating SourceCode. *International Journal of Advanced Computer Science and Applications* 13, 1 (2022), 842 – 853. http://dx.doi.org/10.14569/IJACSA.2022.0130198

[3] Renata Mendes Araujo. 2017. *Information systems and the open world challenges*. SBC, Porto Alegre, RS, Brasil.

[4] V. R. Basili, G. Caldiera, and H. D. Rombach. 1994. The Goal Question Metric approach. *Encyclopedia of Software Engineering* 1, 2 (1994).

[5] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. 2023. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3675–3691. https://doi.org/10.1109/TSE.2023.3267446

[6] Matteo Ciniselli, Luca Pascarella, and Gabriele Bavota. 2022. To What Extent do Deep Learning-based Code Recommenders Generate Predictions by Cloning Code from the Training Set?. In *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*. 167–178. https://doi.org/10.1145/3524842.3528440

[7] Juan Cruz-Benito, Sanjay Vishwakarma, Francisco Martin-Fernandez, and Ismael Faro. 2021. Automated Source Code Generation and Auto-Completion Using Deep Learning: Comparing and Discussing Current Language Model-Related Approaches. *AI* 2, 1 (2021), 1–16. https://doi.org/10.3390/ai2010001

[8] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming (Jack) Jiang. 2023. GitHub Copilot AI pair programmer: Asset or Liability? *Journal of Systems and Software* 203 (2023), 111734. https://doi.org/10.1016/j.jss.2023.111734

[9] Enrique Dehaerne, Bappaditya Dey, Sandip Halder, Stefan De Gendt, and Wannes Meert. 2022. Code Generation Using Machine Learning: A Systematic Review. *IEEE Access* 10 (2022), 82434–82455. https://doi.org/10.1109/ACCESS.2022.3196347

[10] Yogesh K. Dwivedi et al. 2023. Opinion Paper: "So what if ChatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy. *International Journal of Information Management* 71 (2023), 102642. https://doi.org/10.1016/j.ijinfomgt.2023.102642

[11] Yunhe Feng, Sreecharan Vanam, Manasa Cherukupally, Weijian Zheng, Meikang Qiu, and Haihua Chen. 2023. Investigating Code Generation Performance of ChatGPT with Crowdsourcing Social Data. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. 876–885. https://doi.org/10.1109/COMPSAC57700.2023.00117

[12] Dale L Goodhue. 1995. Understanding user evaluations of information systems. *Management science* 41, 12 (1995), 1827–1844.

[13] Dale L Goodhue and Ronald L Thompson. 1995. Task-technology fit and individual performance. *MIS quarterly* (1995), 213–236.

[14] Anjan Karmakar and Romain Robbes. 2021. What do pre-trained code models know about code?. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1332–1336. https://doi.org/10.1109/ASE51524.2021.9678927

[15] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for performing systematic literature reviews in software engineering. (2007).

[16] Sergey Kovalchuk, Dmitriy Fedrushkov, Vadim Lomshakov, and Artem Aliev. 2023. Test-based and metric-based evaluation of code generation models for practical question answering. In *2023 International Conference on Code Quality (ICCQ)*. 73–86. https://doi.org/10.1109/ICCQ57276.2023.10114665

[17] Sila Lertbanjongngam, Bodin Chinthanet, Takashi Ishio, Raula Gaikovina Kula, Pattara Leelaprute, Bundit Manaskasemsak, Arnon Rungsawang, and Kenichi Matsumoto. 2022. An Empirical Evaluation of Competitive Programming AI: A Case Study of AlphaCode. In *2022 IEEE 16th International Workshop on Software Clones (IWSC)*. 10–15. https://doi.org/10.1109/IWSC55060.2022.00010

[18] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. https://doi.org/10.1126/science.abq1158 arXiv:https://www.science.org/doi/pdf/10.1126/science.abq1158

[19] K. Martineau. 2023. What is generative AI? (2023). https://research.ibm.com/blog/what-is-generative-AI

[20] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2149–2160. https://doi.org/10.1109/ICSE48619.2023.00181

[21] Matthew J Page, Joanne E McKenzie, Patrick M Bossuyt, Isabelle Boutron, Tammy C Hoffmann, Cynthia D Mulrow, Larissa Shamseer, Jennifer M Tetzlaff, Elie A Akl, Sue E Brennan, et al. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *Systematic reviews* 10, 1 (2021), 1–11.

[22] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining Zero-Shot Vulnerability Repair with Large Language Models. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2339–2356. https://doi.org/10.1109/SP46215.2023.10179324

[23] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and software technology* 64 (2015), 1–18.

[24] Ilham Qasse, Shailesh Mishra, Björn þór Jónsson, Foutse Khomh, and Mohammad Hamdaqa. 2023. Chat2Code: A Chatbot for Model Specification and Code Generation, The Case of Smart Contracts. In *2023 IEEE International Conference on Software Services Engineering (SSE)*. 50–60. https://doi.org/10.1109/SSE60056.2023.00018

[25] Qureshi. R., D. Shaughnessy, K.A.R. Gill, K.A. Robinson, T. Li, and E. Agai. 2023. Are ChatGPT and large language models "the answer" to bringing us closer to systematic review automation? *Systematic reviews* 12, 72 (2023).

[26] J.S. Sichman. 2021. Inteligência Artificial e sociedade: avanços e riscos. *Estudos avançados. Edição especial: Inteligência Artificial*. 35, 101 (2021). https://doi.org/10.1590/s0103-4014.2021.35101.004

[27] Mohammed Latif Siddiq, Shafayat H. Majumder, Maisha R. Mim, Sourov Jajodia, and Joanna C. S. Santos. 2022. An Empirical Study of Code Smells in Transformer-based Code Generation Techniques. In *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 71–82. https://doi.org/10.1109/SCAM55253.2022.00014

[28] Mohammed Latif Siddiq, Abdus Samee, Sk Ruhul Azgor, Md. Asif Haider, Shehabul Islam Sawraz, and Joanna C. S. Santos. 2023. Zero-shot Prompting for Code Complexity Prediction Using GitHub Copilot. In *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*. 56–59. https://doi.org/10.1109/NLBSE59153.2023.00018

[29] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, and Li Li. 2023. Don't Complete It! Preventing Unhelpful Code Completion for Productive and Sustainable Neural Code Completion Systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 324–325. https://doi.org/10.1109/ICSE-Companion58688.2023.00089

[30] Priyan Vaithilingam, Elena L. Glassman, Peter Groenwegen, Sumit Gulwani, Austin Z. Henley, Rohan Malpani, David Pugh, Arjun Radhakrishna, Gustavo Soares, Joey Wang, and Aaron Yim. 2023. Towards More Effective AI-Assisted Programming: A Systematic Design Exploration to Improve Visual Studio IntelliCode's User Experience. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 185–195. https://doi.org/10.1109/ICSE-SEIP58684.2023.00022

[31] Tim van Dam, Maliheh Izadi, and Arie van Deursen. 2023. Enriching Source Code with Contextual Data for Code Completion Models: An Empirical Study. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. 170–182. https://doi.org/10.1109/MSR59073.2023.00035

[32] Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. 2022. What Do They Capture? - A Structural Analysis of Pre-Trained Language Models for Source Code. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 2377–2388. https://doi.org/10.1145/3510003.3510050

[33] Man-Fai Wong, Shangxin Guo, Ching-Nam Hang, Siu-Wai Ho, and Chee-Wei Tan. 2023. Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review. *Entropy* 25, 6 (2023). https://doi.org/10.3390/e25060888

[34] Chen Yang, Yan Liu, and Changqing Yin. 2021. Recent Advances in Intelligent Source Code Generation: A Survey on Natural Language Based Studies. *Entropy* 23, 9 (2021). https://doi.org/10.3390/e23091174

[35] Ilze Zigurs and Bonnie K Buckland. 1998. A theory of task/technology fit and group support systems effectiveness. *MIS quarterly* (1998), 313–334.